

Task: Building a Responsive Vue 3 Application from Static Layouts

Task Objective:

Convert a provided static HTML template focused on whale conservation into a structured, dynamic Vue 3 application by breaking it down into reusable global and local components. You will implement reactive form handling using Vue's Composition API and create custom CSS styling from scratch. Your finished application must display correctly at a minimum of two viewport sizes: one phone/mobile-sized viewport and one desktop/web-sized viewport.

Task Description:

Use Vite to migrate a static HTML template into a modular Vue 3 application using `<script setup>`. You will break the template down into global and local components, build a responsive layout from scratch using standard CSS and scoped component styles, and implement reactive contact form handling using Vue `ref()` state. Your CSS must support responsive presentation across at least two screen sizes: mobile/phone and desktop/web.

Reference Examples: See Figures 1 and 2 (pages 8–9) for example screenshots showing the expected mobile-sized viewport and desktop-sized viewport layouts.

Task Instructions:

Resources: Use the provided static HTML template and `whale-image.jpg` file supplied in the Resources section on OnTrack. Extract the text content, headings, links, lists, and form structure from the template, then reorganise them into the required Vue components. Use the provided `whale-image.jpg` file as a local asset in your Vue project rather than linking directly to the original external image URL in the template.

Prerequisites: Ensure you have Node.js (version 18 or higher) installed on your computer. Verify by opening your terminal and running:

```
node -v
```

1. Project Setup

- Create a new Vue 3 project using Vite by executing the following commands in your terminal:

```
npm create vite@latest whale-conservation-vue --template vue
cd whale-conservation-vue
npm install
npm run dev
```

- Open your web browser and navigate to the local environment URL:
<http://localhost:5173>.

- Organise your project directory to match the following clean file structure:

```
src/
├── assets/
│   └── whale-image.jpg    <-- Provided in the Resources section
├── components/
│   ├── HeaderComponent.vue
│   ├── FooterComponent.vue
│   ├── MainContent.vue
│   └── ContactForm.vue
├── styles/
│   └── global.css
├── App.vue
└── main.js
```

2. Component Development

A. Global Components (Header & Footer)

These layout-defining components must be registered globally within main.js so they are available for use anywhere in the application.

- `src/components/HeaderComponent.vue`

```
<template>
  <header>
    <!-- TODO: Add header content from the provided HTML template -->
  </header>
</template>

<script setup>
// Component logic can be added here if needed
</script>

<style scoped>
/* TODO: Style the header */
</style>
```

- `src/components/FooterComponent.vue`

```
<template>
  <footer>
    <!-- TODO: Add footer content from the provided HTML template -->
    <!-- Ensure copyright year is updated to 2026 -->
  </footer>
</template>

<script setup>
// Component logic can be added here if needed
</script>

<style scoped>
/* TODO: Style the footer */
</style>
```

- src/main.js

```
import { createApp } from 'vue'
import App from './App.vue'

// Import global stylesheet
import './styles/global.css'

// TODO: Import HeaderComponent and FooterComponent

const app = createApp(App)

// TODO: Register HeaderComponent and FooterComponent globally using
app.component()

// TODO: Mount the application to the target '#app' container defined
in index.html
```

B. Local Components (Main Content & Contact Form)

- src/components/MainContent.vue

```
<template>
  <main>
    <!-- Featured: Whale Conservation Section -->
    <section id="feature">
      <!-- TODO: Add featured section content from the provided HTML
      template -->

      <!-- Dynamic image binding: whaleImg is imported from assets/
      above -->
      
    </section>

    <!-- About Us Section -->
    <section id="about">
      <!-- TODO: Add about section content from the provided HTML
      template -->
    </section>

    <!-- Donate Section -->
    <section id="donate">
      <!-- TODO: Add donate section content from the provided HTML
      template -->
      <!-- NOTE: This specific form uses standard HTML validation
      only; do not add Vue reactivity here -->
    </section>

    <!-- TODO: Render the locally imported ContactForm component here
    -->
  </main>
</template>

<script setup>
// Import the local asset so the Vite compiler asset pipeline
processes it correctly
import whaleImg from '../assets/whale-image.jpg'
```

```
// TODO: Import ContactForm from './ContactForm.vue'
</script>

<style scoped>
/* TODO: Add layout styles specific to MainContent templates */
</style>
```

- `src/components/ContactForm.vue`

```
<template>
  <section id="contact">
    <!-- TODO: Add contact section heading and description -->

    <form @submit.prevent="handleSubmit">
      <!-- TODO: Add form elements for name, email, and message -->
      <!-- Use v-model to bind each distinct form element to your
      reactive state -->

      <!-- TODO: Add standard form submit button -->
    </form>

    <!-- Displays only when showConfirmation is true (use v-show
    directive below) -->
    <div v-show="showConfirmation" class="confirmation-box">
      <p>Thank you, <strong>{{ submittedName }}</strong>! We have
      received your message and will get back to you shortly.</p>
    </div>
  </section>
</template>

<script setup>
import { ref } from 'vue'

// TODO: Create reactive state for a form data property object
// TODO: Create reactive state named 'showConfirmation' for
showing/hiding the confirmation box
// TODO: Create reactive state for tracking the submitted name

function handleSubmit() {
  // TODO: Validate that required fields are completely filled
  // TODO: Capture and store the submitted user name
  // TODO: Toggle the showConfirmation message state to true
  // TODO: Clear all bound form entry properties
}
</script>

<style scoped>
.confirmation-box {
  margin-top: 15px;
  padding: 15px;
  background-color: #d4edda;
  color: #155724;
  border: 1px solid #c3e6cb;
  border-radius: 6px;
}
```

```
/* TODO: Add localised UI presentation styles for your form input
elements */
</style>
```

3. Assembling the Application

- `src/App.vue`

```
<template>
  <div id="app-layout">
    <!-- TODO: Assemble the HeaderComponent, MainContent layout core,
and FooterComponent -->
  </div>
</template>

<script setup>
// TODO: Import the MainContent child layout component
</script>

<style>
#app-layout {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
</style>
```

4. Custom Styling

Write custom, responsive layout structures using standard CSS (Flexbox or Grid elements are encouraged). Use local component `<style scoped>` tag wrappers to safely contain properties that should not contaminate global parent rules.

Your custom CSS must produce a responsive layout that works at a minimum of two viewport sizes: one phone/mobile view and one desktop/web view. You must use responsive CSS techniques such as media queries, Flexbox, Grid, or other appropriate layout strategies to ensure the interface remains usable and visually clear at both sizes.

- `src/styles/global.css`

```
/* TODO: Add a universal box-sizing reset rule */
/* TODO: Add base body styles (font-family, line-height, color,
background-color) */
/* TODO: Add mobile-first base spacing and layout */

/* TODO: Add anchor link and button base styles */
/* TODO: Add heading hierarchy styles (h1, h2, h3) */
/* TODO: Add form input, textarea, and label base styles */

/* TODO: Add responsive media query for desktop (1024px and above) */
/* @media (min-width: 1024px) {
  TODO: Add desktop-specific adjustments to spacing, layout, typography
} */
```

5. Video Demonstration Requirements

You must submit a link deposited to your choice of provider (**Deakin OneDrive**, Google Drive, or YouTube) to a **3-minute video recording** walking through your completed application. The link must be fully accessible to both your **marker** and the **Unit Chair**, remaining active until official results are released.



Recording Requirements:

- **Visibility:** Your **face must be visible** on camera (via webcam picture-in-picture) throughout the video, alongside **clear spoken narrative**. Screen-only captures or muted recordings without your face will not be accepted and will require resubmission of this component.

Student with Access Plan: If you cannot provide video with visible face and/or audio, please contact your Unit Chair before the submission due date to arrange an alternative

- **Part 1: Code Walkthrough (approx. 1.5 mins):** Open your code editor and briefly show your `main.js` configuration, your component file structure, and how you implemented the dynamic form states (`v-model`, `v-show`, and `handleSubmit`) inside `ContactForm.vue`.
- **Part 2: Browser Demonstration (approx. 1.5 mins):** Show your live application running in the browser at <http://localhost:5173>. Demonstrate the layout's responsiveness by resizing the window, fill out the donation form to show browser validation, and submit the contact form to demonstrate the reactive confirmation message appearing and the fields clearing instantly. Your video must clearly show both a phone/mobile-sized viewport and a desktop/web-sized viewport.

Deliverables:

You are required to submit only the specific files you created or edited to complete this task, along with the link to your video and two browser screenshots.

You must submit the following:

- **7 source files** you created or edited to complete the task (listed below).
- **Two screenshots** of your webpage showing the Vue implementation in the browser: one in a desktop/web-sized viewport and one in a phone/mobile-sized viewport.
- A working and **fully accessible link** to your 3 minute **video demonstration** must be provided to **both your marker and the Unit Chair**. The link must remain accessible until official results are released.

Source Files:

1. **main.js** — Global component registration and application mounting logic.
2. **App.vue** — Core layout assembly bringing all sections together.
3. **HeaderComponent.vue** — Extracted header markup and localised styling.

4. **FooterComponent.vue** — Extracted footer markup and localised styling.
5. **MainContent.vue** — Content container handling local component imports.
6. **ContactForm.vue** — Reactive form state, validation, and Composition API logic.
7. **global.css** — Your custom global layout resets and baseline element styling.

Quick Checklist:**Source Files (7 required)**

- ☐ main.js — global component registration
- ☐ App.vue — layout assembly
- ☐ HeaderComponent.vue — header with styling
- ☐ FooterComponent.vue — footer (copyright 2026)
- ☐ MainContent.vue — sections + local image import
- ☐ ContactForm.vue — reactive form with v-model, v-show, validation
- ☐ global.css — responsive styles + media queries

Responsive Design

- ☐ Works on mobile/phone-sized viewport
- ☐ Works on desktop/web-sized viewport

Functionality

- ☐ Contact form uses ref() for reactive state
- ☐ Contact form fields bound with v-model
- ☐ Contact form submits with @submit.prevent
- ☐ Contact form validation: required fields check
- ☐ Contact form shows confirmation message (v-show)
- ☐ Contact form clears fields after submission
- ☐ Donation form uses HTML validation only (no Vue reactivity)
- ☐ Local image (whale-image.jpg) displays (not external URL)
- ☐ Application runs without errors at http://localhost:5173

Screenshots (2 required)

- ☐ Mobile view screenshot
- ☐ Desktop view screenshot

Video (3 mins, with your face visible)

- ☐ Part 1 (~1.5 min): Show code — main.js, components, form logic

- ☐ Part 2 (~1.5 min): Show app running, resize to mobile/desktop, test form submission
- ☐ Link is public and accessible to **marker AND Unit Chair**

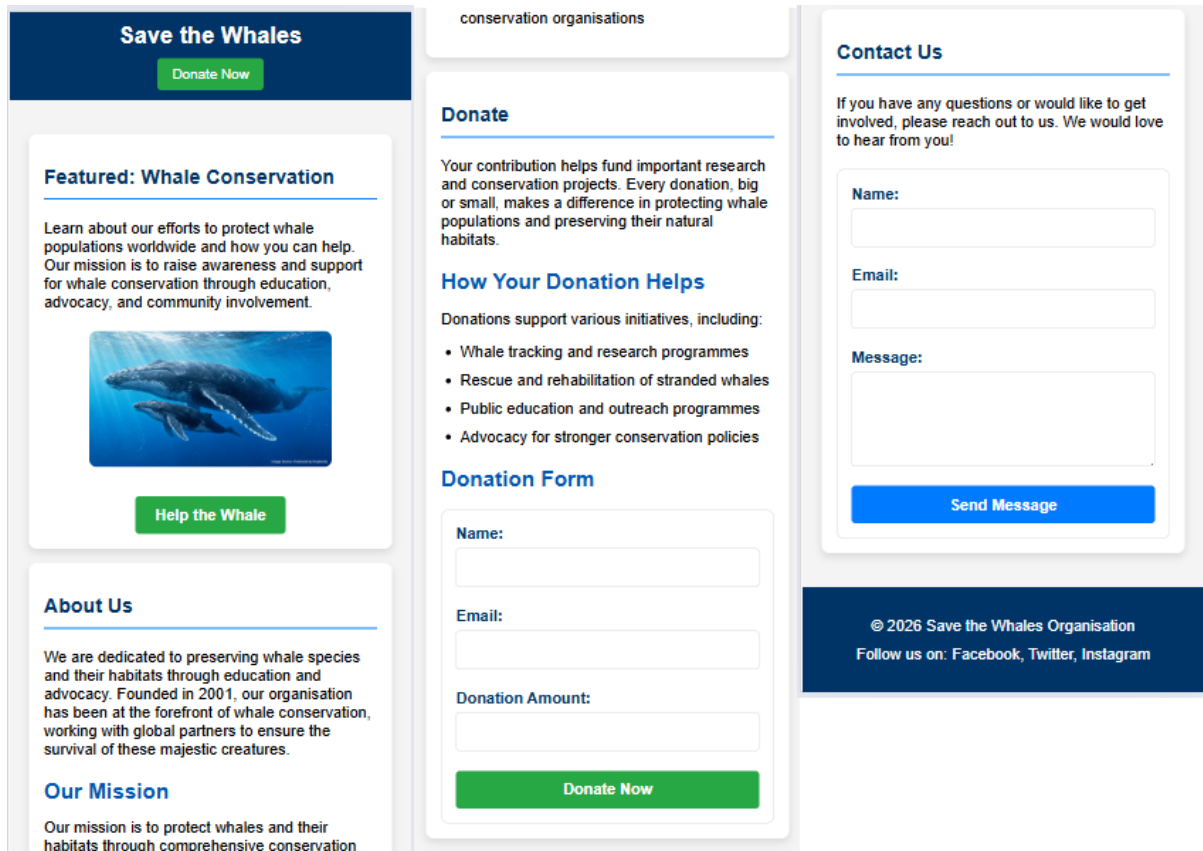
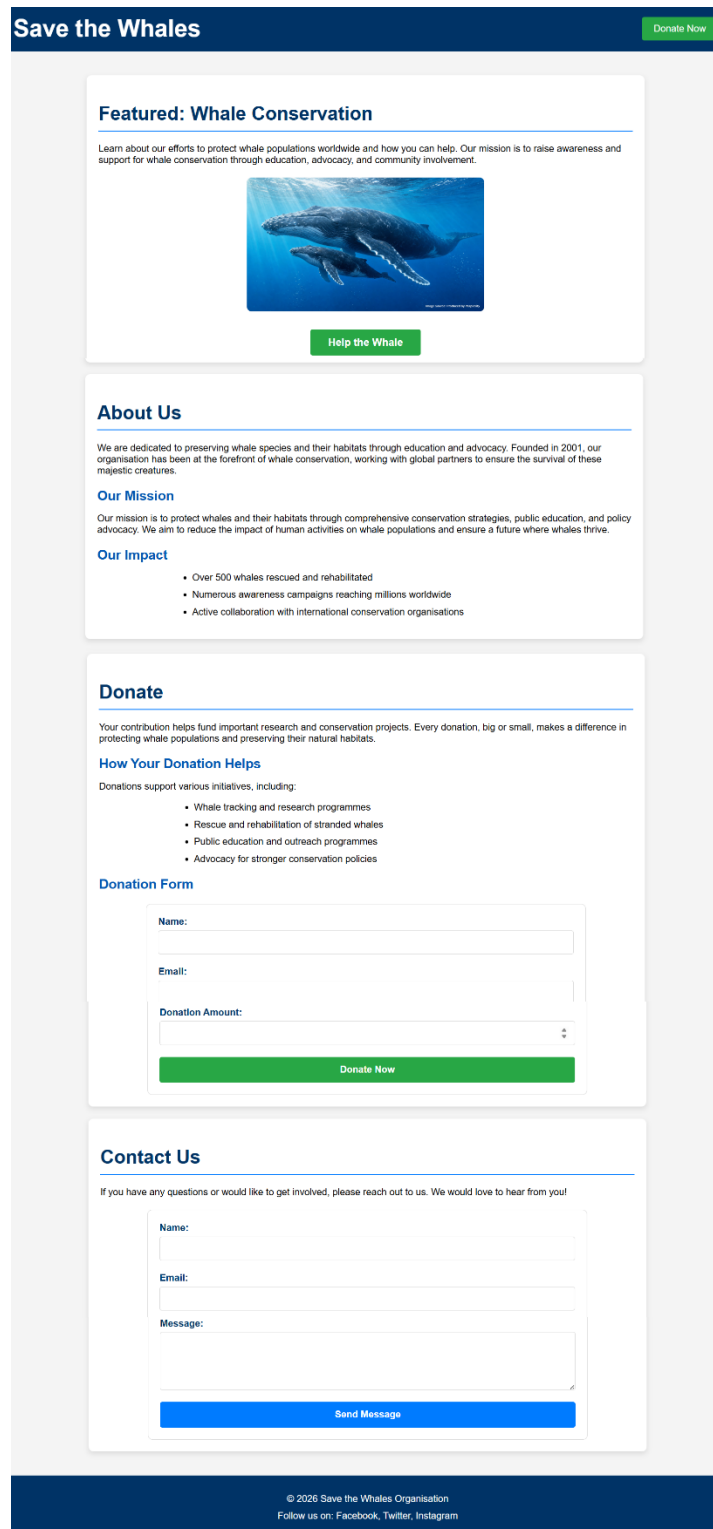


Figure 1: Mobile View

Screenshot of the Save the Whales application rendered on a mobile/phone-sized viewport, displaying the responsive layout with stacked navigation, constrained image width, and full-width form elements.

**Figure 2: Desktop View**

Screenshot of the Save the Whales application rendered on a desktop/web-sized viewport, displaying the enhanced layout with side-by-side navigation, expanded image dimensions, and optimised form presentation with wider maximum width.